Term Project #1

Project Description and Architecture Decision Records

Project Name: Tutor Matcher

Group Members

- Kritarat Moonmanee
- Teerachot Kerdlapanan
- Piranat Wadlom
- Nathawat Wattanarapeepong

1. Project Proposal

Problem Description and Solution

Finding a tutor who matches a student's subject, schedule and budget often requires separate searches, messages and payment arrangements. Students need clear availability and prices, while tutors need a reliable schedule and a record of their earnings. Tutor Matcher brings tutor discovery, one-to-one booking, online lesson access and wallet payments into one web application.

Students book a specific subject using a continuous block of a tutor's published 30-minute slots. Wallet payment confirms the booking immediately. Tutor verification, subject-specific reviews and administrative dispute handling support trust between participants.

Target Users

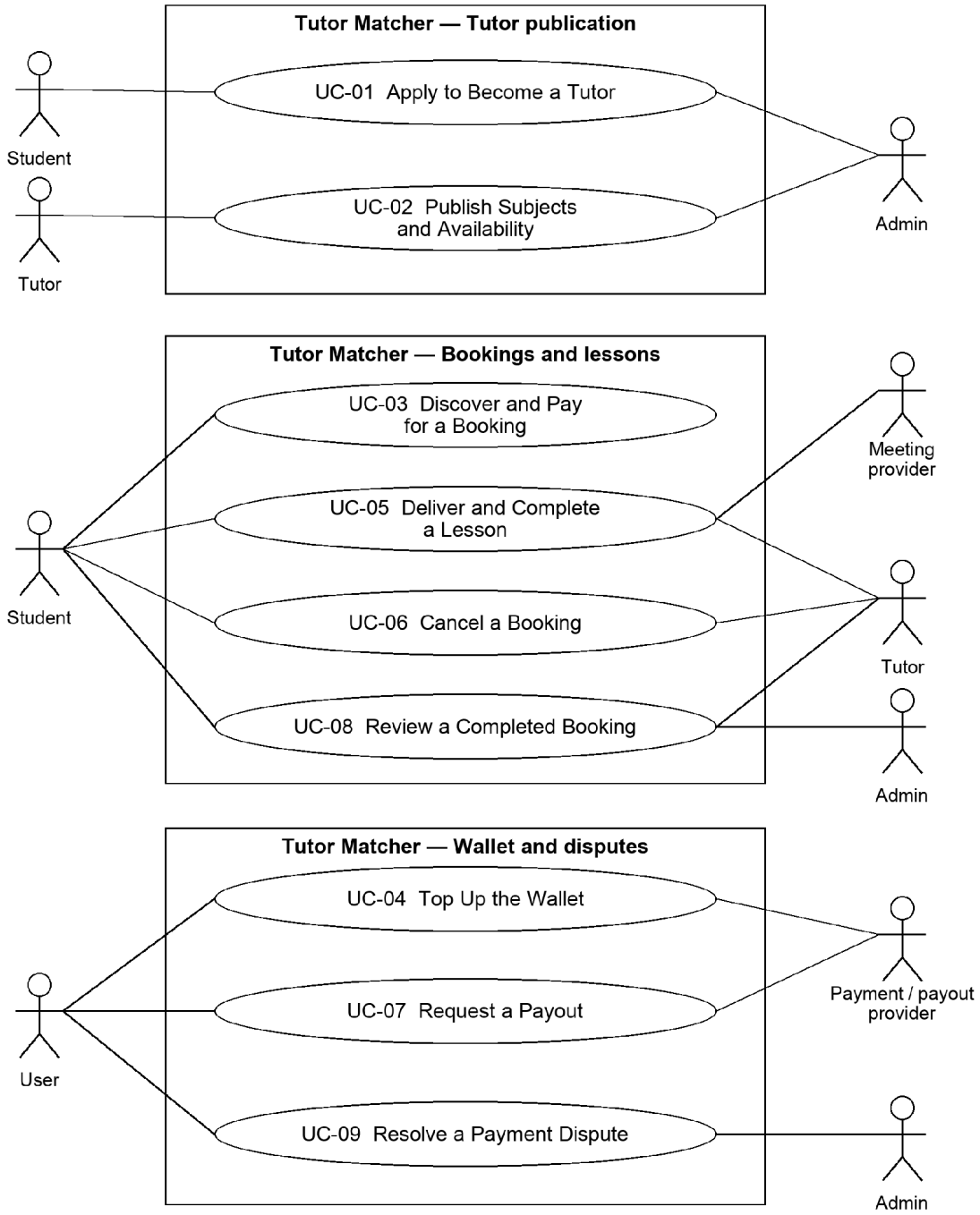
Student: Find tutors, book and attend lessons, message tutors, review completed bookings, and manage wallet funds.

Tutor: Publish subjects and availability, teach booked lessons, and receive earnings. Tutor capability is added to the same account after an application is approved.

Admin: Review tutor applications and listing changes, moderate content, resolve payment disputes, and manage account restrictions.

2. Use Case Diagram

The three views below show the same Tutor Matcher system. Actors outside each boundary interact with the use cases inside it. “User” means any registered account, including a student or tutor; one person can participate in more than one role.



Solid lines show participation, not execution order. Top-up is a separate user goal that can be reached from booking when funds are insufficient.

3. Use Case Descriptions

The following nine main use cases describe actors, goals, system interactions and outcomes. Supporting actors participate in a flow without necessarily initiating it.

Use Case Name: Apply to Become a Tutor	**ID:** UC-01	**Important Level:** High
Primary Actor: Student; supporting actor: Admin		**Use Case Type:** Business / Approval
Stakeholders and Interests: Student: gain permission to teach. Admin: verify eligibility.		
Brief Description: A student submits an application and receives a review outcome.		
Trigger: The student selects Become a tutor. Type: External		
Relationships: Admin participates in the application review.		
Precondition: The student is signed in and can provide the required documents and bio.		
Postcondition: Approval grants tutor capability to the same account; rejection records a reason.		
Normal Flow of Events: ● The student submits a government ID document, teaching certification and bio. ● The system validates the submission and displays its pending status. ● An admin examines the application and approves or rejects it. ● The system records the decision and informs the applicant.		
Subflows: A student can cancel a pending application and submit a replacement. **Alternate/Exceptional Flow:** Missing required information prevents submission. A rejected application does not grant tutor capability.		

Use Case Name: Publish Subjects and Availability	**ID:** UC-02	**Important Level:** High
Primary Actor: Tutor; supporting actor: Admin		**Use Case Type:** Business / Publication
Stakeholders and Interests: Tutor: advertise lessons and teaching times. Student: see bookable offerings.		
Brief Description: An approved tutor maintains a public listing, subjects and dated availability.		
Trigger: The tutor opens the teaching management area. Type: External		
Relationships: Admin reviews listing and verification-document changes.		
Precondition: The signed-in user has tutor capability.		
Postcondition: Students can discover approved listing content and the tutor's offered subjects and available slots.		
Normal Flow of Events: ● The tutor submits listing or verification-document changes; the system routes them to admin review. ● The admin approves or rejects the changes; approved content becomes public. ● The tutor enters subject descriptions, hourly rates and lesson formats. ● The tutor opens dated 30-minute slots and assigns one or more subjects to each slot.		
Subflows: Existing approved listing content stays visible while a replacement is being reviewed. **Alternate/Exceptional Flow:** Invalid subject or slot details are rejected. The tutor cannot close a slot already held by a paid booking.		

3. Use Case Descriptions (continued)

Use Case Name: Discover and Pay for a Booking	**ID:** UC-03	**Important Level:** High
Primary Actor: Student		**Use Case Type:** Business / Transactional
Stakeholders and Interests: Student: secure a suitable lesson at a clear price. Tutor: avoid overlapping bookings.		
Brief Description: A student finds a tutor's subject, selects continuous slots and pays from the wallet.		
Trigger: The student searches for a tutor or opens a subject. Type: External		
Relationships: UC-04 provides top-up when the wallet has insufficient funds.		
Precondition: A tutor has published a subject and availability; the student signs in before creating a booking.		
Postcondition: The booking is confirmed with its subject, schedule and price fixed, and appears on both participants' schedules.		
Normal Flow of Events: ● The student filters tutors, opens a subject and chooses back-to-back 30-minute slots. ● The system shows duration and price and accepts a note about the learning goal. ● The student continues; the system creates a payment-due booking and holds the selected slots. ● The student pays; the system deducts wallet funds and confirms the booking immediately.		
Subflows: If funds are short, the student tops up and returns to the same booking. No tutor acceptance step is required. **Alternate/Exceptional Flow:** Unavailable or non-adjacent slots cannot be booked. Insufficient balance blocks Pay. Repeated payment requests must not cause a second debit.		

Use Case Name: Top Up the Wallet	**ID:** UC-04	**Important Level:** High
Primary Actor: User; supporting actor: Payment provider		**Use Case Type:** Business / Payment
Stakeholders and Interests: User: add spendable funds. Platform: credit only confirmed payments.		
Brief Description: A user adds money to the single wallet through PromptPay.		
Trigger: The user selects Top up, directly or from a booking shortfall. Type: External		
Relationships: Can return the user to UC-03 after a booking-related top-up.		
Precondition: The user is signed in and enters a valid amount.		
Postcondition: A successful payment increases available balance once and appears in transaction history.		
Normal Flow of Events: ● The user selects a preset amount or enters a custom amount. ● The system shows a PromptPay QR code and payment details. ● The user pays through a banking application; the payment provider returns the outcome. ● The system verifies confirmation, credits the wallet and displays the updated balance.		
Subflows: A booking-related top-up returns to the booking so the user can complete lesson payment. **Alternate/Exceptional Flow:** Invalid amounts are rejected. Failed or unconfirmed payments do not increase balance. Repeated confirmations do not create duplicate credits.		

3. Use Case Descriptions (continued)

Use Case Name: Deliver and Complete a Lesson	**ID:** UC-05	**Important Level:** High
Primary Actor: Student and Tutor; supporting actor: Meeting provider		**Use Case Type:** Business / Fulfillment
Stakeholders and Interests: Participants: attend the booked lesson. Tutor: receive earnings. Student: become eligible to review.		
Brief Description: The participants join a confirmed online lesson and the system records completion.		
Trigger: A participant opens Join for the scheduled lesson. Type: External		
Relationships: Completion enables UC-08; cleared earnings become available for UC-07.		
Precondition: The online booking is confirmed and the caller is one of its participants.		
Postcondition: A completed booking creates pending tutor earnings and enables the student to review it.		
Normal Flow of Events: ● The system shows the confirmed lesson details and authorized meeting link. ● The student and tutor join the room supplied by the meeting provider. ● The system obtains attendance evidence and checks the completion rules. ● The system marks an eligible booking completed, records pending earnings and enables its review action.		
Subflows: Pending earnings become available after the settlement clearing period. **Alternate/Exceptional Flow:** Joining early shows the room-opening information. Missing attendance evidence is flagged for review. A cancelled booking cannot become completed.		

Use Case Name: Cancel a Booking	**ID:** UC-06	**Important Level:** High
Primary Actor: Student or Tutor		**Use Case Type:** Business / Adjustment
Stakeholders and Interests: Participants: understand the cancellation and its financial effect.		
Brief Description: A participant cancels a booking after seeing the applicable consequences.		
Trigger: The participant selects Cancel on a booking. Type: External		
Relationships: A disputed payment outcome can be handled through UC-09.		
Precondition: The user is a participant in a booking eligible for cancellation.		
Postcondition: The booking is cancelled, removed from upcoming lessons, and any eligible refund is recorded in the student's wallet.		
Normal Flow of Events: ● The participant opens the booking and requests cancellation. ● The system shows the applicable cancellation policy and refund consequence. ● The participant confirms; the system records cancellation and removes lesson access. ● The system records any eligible refund and updates both participants' booking views.		
Subflows: The cancelled booking and its payment history remain available for reference. **Alternate/Exceptional Flow:** An unauthorized or ineligible request is rejected. A refund still being processed is shown as pending and must not be duplicated.		

3. Use Case Descriptions (continued)

Use Case Name: Request a Payout	**ID:** UC-07	**Important Level:** High
Primary Actor: User; supporting actor: Payout provider		**Use Case Type:** Business / Transactional
Stakeholders and Interests: User: withdraw available money to a saved account.		
Brief Description: A user requests a transfer from available wallet balance to a payout account.		
Trigger: The user selects Request payout. Type: External		
Relationships: Uses available funds, including successful top-ups and cleared earnings.		
Precondition: The user is signed in, has a saved payout account and has sufficient available balance.		
Postcondition: The payout outcome and resulting balance are recorded in transaction history.		
Normal Flow of Events: ● The user chooses a payout amount and saved destination account. ● The system validates the amount and reserves it against further spending. ● The system submits the transfer to the payout provider. ● The system records the provider outcome and shows the payout status to the user.		
Subflows: Both students and tutors can request payouts. Pending earnings are excluded from available funds. **Alternate/Exceptional Flow:** An amount above available balance is rejected. An unknown provider outcome remains pending; a definitively failed payout restores the reserved funds.		

Use Case Name: Review a Completed Booking	**ID:** UC-08	**Important Level:** High
Primary Actor: Student; supporting actors: Tutor and Admin		**Use Case Type:** Business / Reputation
Stakeholders and Interests: Student: share experience. Tutor: respond to feedback. Admin: handle flagged content.		
Brief Description: A student publishes feedback about a completed lesson with a specific tutor and subject.		
Trigger: The student selects Review on a completed booking. Type: External		
Relationships: Requires the completed booking outcome from UC-05.		
Precondition: The student owns the completed booking and has not already reviewed it.		
Postcondition: The review is linked to the booking's tutor and subject and contributes to visible rating summaries.		
Normal Flow of Events: ● The system verifies that the booking is completed and belongs to the student. ● The student enters a rating from 1 to 5 and review text, with an optional headline. ● The system publishes the review under the relevant tutor and subject. ● The tutor may reply; flagged content is routed to an admin for a recorded moderation decision.		
Subflows: Users may flag a review with a reason. An admin may keep, hide or remove flagged content. **Alternate/Exceptional Flow:** Incomplete bookings, another student's booking and duplicate reviews are rejected. The same user cannot flag the same review twice.		

3. Use Case Descriptions (continued)

Use Case Name: Resolve a Payment Dispute	**ID:** UC-09	**Important Level:** High
Primary Actor: Admin; supporting actor: User		**Use Case Type:** Business / Case handling
Stakeholders and Interests: Affected user: receive an explained outcome. Admin: resolve the case with an audit trail.		
Brief Description: An admin investigates a reported payment dispute and records the remedy.		
Trigger: A user reports a payment issue and a dispute enters the admin queue. Type: External		
Relationships: May concern payment in UC-03 or a cancellation outcome in UC-06.		
Precondition: A dispute identifies the affected booking or payment; the admin is authorized to handle it.		
Postcondition: The dispute has a recorded decision and any authorized financial adjustment appears in wallet history.		
Normal Flow of Events: ● The user provides the payment issue and relevant booking details. ● The admin reviews the case, booking state and transaction history. ● The admin records a decision, reason and any refund instruction. ● The system applies the authorized adjustment once and makes the outcome available to the affected user.		
Subflows: A case can be resolved without a refund when the evidence does not justify a financial adjustment. **Alternate/Exceptional Flow:** Missing evidence requires further review. An adjustment that cannot yet complete remains pending without duplicating the financial effect.		

Supporting Capabilities

Registration and login establish account access. Messaging lets a student contact a tutor before or after booking. Notifications communicate application, booking, lesson and wallet updates according to user preferences. These capabilities support the main use cases and are included in the system scope.

4. Functional Requirements

Accounts and Tutor Publication

- The system shall create a student-capable account and support email/password login, Google login, logout and password reset.
- The system shall grant tutor capability after application approval and review listing and verification-document changes before publication.
- Tutors shall manage subjects with individual rates, descriptions and formats, and dated 30-minute availability with subject assignments.

Discovery and Booking

- Students shall search by keyword and filter by subject, price, rating and format.
- A booking shall contain one student, one tutor, one subject and continuous adjacent slots. The total price shall equal the number of half-hour slots multiplied by half the hourly rate.
- The system shall hold payment-due slots and confirm a booking immediately after wallet payment, without tutor acceptance.
- The system shall preserve the booking context through top-up and prevent overlapping sale and duplicate lesson payment.

Wallet and Lessons

- Each user shall have one wallet with available balance, pending earnings and read-only transaction history.
- The system shall support PromptPay top-ups, lesson payments, earnings, refunds and payouts to saved accounts.
- Confirmed online bookings shall provide meeting access to participants. Completion shall enable pending earnings and review eligibility.

Trust and Communication

- Participants shall be able to cancel eligible bookings after seeing the applicable refund consequence.
- Students shall review their own completed bookings; tutors may reply and admins may moderate flagged content.
- The system shall support messaging, notification preferences, administrative dispute resolution and account restrictions.

5. Non-Functional Requirements

Operational

- Use separate local and production deployment configurations and persistent PostgreSQL storage.
- Encrypt deployment secrets and test database backup and recovery procedures.
- Register every service in a service registry and check its health automatically, so a crashed instance leaves rotation without manual action.

Performance and Reliability

- Prevent double-booking, double-spending and duplicate credits during concurrent requests and retries.
- Retain payment and payout attempts until their external outcomes can be reconciled.
- Keep notification or meeting-provider delays from reversing a successful lesson payment.
- Scale tutor search horizontally: with 50 concurrent users, adding Tutor Service replicas must raise throughput while the 95th-percentile latency stays under 500 ms and errors stay under 1%.
- Keep the gateway serving requests when one Tutor Service instance fails or when the service registry is unavailable.

Security

- Enforce account permissions and resource ownership for protected actions.
- Restrict access to verification documents, payout details and meeting links.
- Verify external payment confirmations and retain an audit trail for administrative financial actions.

Cultural and Privacy Considerations

- Display wallet amounts in Thai baht and show dates and times consistently throughout the booking journey.
- Collect agreement to Terms and Privacy Policy during registration and restrict personal data to its intended operational use.

Usability

- Display field-level validation errors, payment status and cancellation consequences clearly.
- Keep payment-due, confirmed, completed and cancelled bookings distinguishable.
- Distinguish spendable balance from pending earnings and preserve the user’s destination through login and top-up.

Maintainability and Testability

- Keep frontend presentation separate from backend business rules and data access.
- Test booking lifecycle transitions, financial transactions, authorization and external-provider recovery.
- Automate formatting, static checks, tests and application builds.
- Keep a repeatable load test in the repository and save its results with the code.

6. Architecture Decision Records (ADRs)

ADR-001: Next.js for Presentation and Routing

Status: Accepted.

Context: Tutor Matcher needs a web interface for browsing tutors, booking lessons and managing a wallet. Rendering and page routing are separate concerns from booking and wallet rules.

Decision: Build the web interface with Next.js. It owns pages, navigation and rendering only. It holds no booking, wallet or access rules; it calls the backend for them.

Consequences:

- Interface changes ship without touching business rules.
- Every rule the interface needs must be exposed by the backend API.
- Interface tests can run against a mocked API.

ADR-002: Separate Express Backend in TypeScript

Status: Accepted.

Context: Booking, wallet and access rules must behave consistently for every client. They have different responsibilities and testing needs from the interface.

Decision: Implement business behavior in a standalone Express backend written in TypeScript. The backend is the only component that enforces booking, wallet and access rules.

Consequences:

- The API can be tested independently of the interface.
- Frontend and backend communicate through explicit request and response contracts.
- Two runtimes must be built, configured and operated.

ADR-003: Prisma with PostgreSQL for Data Access

Status: Accepted.

Context: The backend needs a typed, maintainable way to read and write booking, wallet and account data with transactional guarantees.

Decision: Use Prisma as the data-access layer over PostgreSQL. All reads and writes of booking, wallet and account data go through the Prisma client.

Consequences:

- The data model is defined once in a typed Prisma schema, and queries are type-checked.
- Booking and wallet changes can run in a single database transaction.
- Schema changes need reviewed migrations, and PostgreSQL must be operated and backed up.

6. Architecture Decision Records (continued)

ADR-004: Monorepo with npm Workspaces

Status: Accepted.

Context: The frontend and backend evolve together. A booking or wallet change can require coordinated interface, API and test updates, so one change should be able to touch both.

Options considered.

Option	Description	Verdict
Polyrepo	One repository per application	Rejected: cross-cutting changes need linked pull requests and version coordination
Monorepo with npm workspaces	One repository, `frontend/` and `backend/` as workspace packages, one lockfile	Selected
Monorepo with a build orchestrator (Turborepo, Nx)	Workspaces plus task caching and pipelines	Deferred: adds tooling the project does not yet need

Decision: Develop in a monorepo using npm workspaces. The root holds shared formatting, linting and test configuration and one lockfile. Each package keeps its own `package.json`, build script, runtime configuration and responsibilities. Root scripts (such as `npm run build`) run the matching script in every package.

Consequences:

- One pull request can change the interface, API and tests together.
- Shared tooling reduces configuration drift.
- Dependency or build-setting changes at the root affect both packages and need coordination.
- Adopt a build orchestrator later if build times justify it.

6. Architecture Decision Records (continued)

ADR-005: Separate Docker Compose Files for Local and Production

Status: Accepted.

Context: Local development needs fast startup, hot reload and debugging. Production needs fixed images and controlled configuration. One file serving both would mix the two.

Decision: Keep two Docker Compose configurations: a local file that builds from source with mounted code and debug settings, and a production file that runs published images only.

Consequences:

- Developers get a consistent local application stack with one command.
- Production never builds from source or mounts local code.
- The two files must be kept aligned on service names and ports.

ADR-006: Build and Publish Images with GitHub Actions

Status: Accepted.

Context: Production must run images that are built the same way every time and traceable to a commit.

Decision: Build the frontend and backend images in GitHub Actions and publish them to a container registry, tagged by commit. Production pulls these images; it never builds them.

Consequences:

- Every deployed image is repeatable and traceable to a commit.
- The build pipeline and registry credentials become part of the supply chain to protect.
- A failed build blocks release.

ADR-007: Deploy the Production Swarm Stack with Doco-CD

Status: Accepted.

Context: Production changes should be applied from a reviewed source of truth, not by hand on the server.

Decision: Run production as a Docker Swarm stack and use Doco-CD to deploy it from the repository's production Compose configuration when it changes.

Consequences:

- Deployments are declarative, repeatable and auditable through Git history.
- Rollback is a revert of the configuration.
- Doco-CD becomes a deployment dependency that must be run and monitored.

6. Architecture Decision Records (continued)

ADR-008: SOPS and age for Environment Secrets

Status: Accepted.

Context: Production configuration includes secrets. They must be versioned with the deployment configuration without being readable in the repository.

Decision: Encrypt environment secrets with SOPS using age keys and commit the encrypted files. Doco-CD decrypts them at deploy time.

Consequences:

- Secret changes are reviewed and versioned like other configuration.
- Plaintext secrets never enter the repository.
- The age private key is a critical asset: its storage, rotation and recovery need an owner.

ADR-009: Service Extraction Path

Status: Accepted.

Context: The decision to run every bounded context inside one Express deployment can read as a commitment to a monolith. The intent is different. The shared deployment is the first step, and contexts move out of it individually once evidence shows that separation is worth its cost.

Decision: Treat deployment topology as per-context and evidence-driven. Keep every context in the shared backend until a specific one meets an extraction gate. Extract a context when independent scaling, failure isolation or independent release ownership is demonstrably needed, and record the measured benefit rather than asserting it. Extract in dependency order, beginning with contexts that already communicate through events; Booking and Wallet extract last because they share a deliberate payment transaction boundary. Every extraction ships versioned contracts, owner-specific persistence, replay and recovery procedures, and monitoring before the old path is removed.

Consequences:

- The single deployment is bounded in time and scope, so the architecture has a stated direction instead of an implied one.
- No extraction proceeds on architectural preference alone; each one requires its own evidence and its own accepted decision.
- Domain boundaries must stay clean inside the shared backend, because they are the units that are extracted later.
- Follow-ups: ADR-010 (Tutor service) and ADR-011 (Booking service).

6. Architecture Decision Records (continued)

ADR-010: Extract the Tutor Service

Status: Accepted.

Context: All bounded contexts run in one Express deployment. Tutor (profiles, subjects, availability, search) is read-heavy and communicates with other contexts through events, so it meets the extraction order's starting condition.

Decision: Extract Tutor into its own service once at least one extraction gate is measured: independent scaling, failure isolation or independent release ownership. Record the measured benefit in this ADR before the old path is removed. The extraction must ship:

- versioned API and event contracts;
- persistence owned only by the Tutor service;
- replay and recovery procedures for events;
- monitoring in place before the old in-process path is removed.

Consequences:

- Tutor can scale and release independently of the rest of the backend.
- Other contexts depend on Tutor only through its contracts, never its tables.
- Operating a second service adds deployment, monitoring and on-call cost.
- Extraction proceeds only when its evidence is recorded.

ADR-011: Extract the Booking Service

Status: Accepted.

Context: Booking shares a deliberate payment transaction boundary with Wallet, so it is extracted last. Today a booking is confirmed and the wallet debited in one database transaction. Separating Booking breaks that boundary unless the replacement is designed first.

Decision: Extract Booking after Tutor (ADR-010), and only once an extraction gate is measured and recorded. Before the old path is removed, the extraction must ship:

- versioned contracts with Tutor, Wallet and Fulfillment;
- persistence owned only by Booking;
- a replacement for the single transaction, with durable payment attempts and one operation identity per request so retries cannot duplicate charges;
- reconciliation of uncertain payment outcomes before held slots are released;
- replay and recovery procedures, and monitoring.

Consequences:

- Booking can scale and release independently.
- The atomic booking-and-payment guarantee becomes an explicit protocol that needs its own tests.
- Payment timeouts require reconciliation, which adds operational work.
- Wallet extraction, if ever needed, becomes a separate decision.

ADR-012 to ADR-020 record the service extraction decisions made under ADR-009 and implemented in the microservice version (tutor-matcher-microservice). Tutor and Booking extraction are ADR-010 and ADR-011.

ADR-012: Extract the Notification Service

Status: Accepted.

Context: ADR-009 extracts contexts that already communicate through events first. Notification only reacts to events from other contexts, and a notification failure must never fail a booking or top-up.

Decision: Extract Notification first, under ADR-009's failure-isolation gate. It consumes events and owns its delivery log. Evidence: with Notification stopped, a wallet top-up still succeeded, and the queued event was delivered after restart.

Consequences:

- A Notification outage no longer affects business actions.
- Notification depends on the message broker (ADR-019) and must ignore duplicate events.

ADR-013: Extract the Auth Service

Status: Accepted.

Context: Every protected use case needs a signed-in user. Credentials need stricter isolation than other data, and Auth communicates with other contexts only through events and issued tokens.

Decision: Extract Auth after Notification, under ADR-009's failure-isolation and independent-release gates. Auth owns accounts and password hashes, issues JWTs, and publishes an event when an account is registered.

Consequences:

- No other service reads credentials; they receive the user identity from the gateway (ADR-018).
- Tokens are stateless, so a token stays valid until it expires.

ADR-014: Extract the Wallet Service

Status: Accepted.

Context: ADR-009 extracts Wallet last because it shares a payment transaction boundary with Booking. Money movements need their own consistency guarantees.

Decision: Extract Wallet last, after Booking (ADR-011), under ADR-009's failure-isolation and independent-release gates. Wallet owns balances, top-ups, payments and refunds, and exposes charge and refund only as internal endpoints.

Consequences:

- Booking can no longer confirm and pay in one database transaction, so ADR-011's payment safeguards apply (see ADR-020).
- Wallet must be available for bookings to be paid or cancelled.

ADR-015: Database per Service

Status: Accepted.

Context: ADR-003 gives the single backend one shared PostgreSQL database accessed through Prisma. Extracted services (ADR-010 to ADR-014) cannot share it without coupling their schemas and releases.

Decision: Each extracted service owns a separate database that only it reads and writes. Other services get its data only through its API or its events. Each service chooses the database engine that fits its data, recorded in ADR-016 and ADR-017. ADR-003 still applies to the single-backend version.

Consequences:

- A service can change its schema and deploy without coordinating with other services.
- There are no cross-service joins or transactions; shared data moves through APIs or events (ADR-020).
- There are more databases, possibly on different engines, to run, back up and monitor.

ADR-016: PostgreSQL for Auth and Wallet

Status: Accepted.

Context: Accounts and money need strict consistency: unique emails, balances that never go negative, and no payment applied twice, even under concurrent requests.

Decision: Auth and Wallet each use their own PostgreSQL database (auth_db and wallet_db). Wallet relies on transactions, row locks, a non-negative balance check and a unique key per payment reference.

Consequences:

- The database itself enforces unique accounts, correct balances and no duplicate charges or refunds.
- Concurrent payments to one wallet wait on row locks.
- Schema changes need migrations.

ADR-017: MongoDB for Tutor, Booking and Notification

Status: Accepted.

Context: Tutor profiles, availability slots and bookings, and the notification delivery log are self-contained documents that are read far more often than they are joined.

Decision: Tutor, Booking and Notification each use their own MongoDB database (tutor_db, booking_db and notification_db). Booking holds a slot with an atomic conditional update, and Notification keeps a unique index on the event ID.

Consequences:

- Document shapes can change without migrations.
- A slot cannot be double-booked, and a redelivered event is not sent twice.
- Holding several slots is not one transaction, so a failed hold must release the slots already taken.
- Rules a relational schema would enforce must be checked in code.

ADR-018: API Gateway with Centralized Authentication

Status: Accepted.

Context: With several services, clients should not know each service's address, and each service should not verify logins on its own.

Decision: Put an API gateway in front of all services as the single entry point. It verifies the JWTs issued by Auth (ADR-013) and passes the signed-in user to downstream services. Service-to-service endpoints are not exposed through the gateway.

Consequences:

- Clients use one address, and token checks live in one place.
- Services trust the identity the gateway passes, so they must not be reachable from outside.
- The gateway becomes a critical component that must stay available.

ADR-019: RabbitMQ as the Message Broker

Status: Accepted.

Context: Extracted services publish a small number of domain events that must not be lost while a consumer is down. Options considered: RabbitMQ, Apache Kafka and Redis Pub/Sub.

Decision: Use RabbitMQ. Durable queues with acknowledgements hold events until a consumer processes them, and a topic exchange routes each event type to its subscribers. Kafka's retained log and partitioning are more than this event volume needs and cost more to operate. Redis Pub/Sub drops messages when no consumer is connected.

Consequences:

- Events survive consumer outages and broker restarts.
- Delivery is at least once, so consumers must ignore duplicates.
- A consumed event is not kept, so replaying past events (ADR-009) needs another source, such as an outbox.
- RabbitMQ is another component to run and monitor.

ADR-020: Synchronous Calls for Payment, Asynchronous Events for Side Effects

Status: Accepted.

Context: Booking and Wallet no longer share a database transaction. A booking must be confirmed only after payment succeeds, while notifications must never block or fail a business action.

Decision: Booking calls Wallet synchronously to charge and refund, using the per-request operation identity that ADR-011 requires. Services publish domain events (account registered, wallet topped up, booking confirmed, booking cancelled) through RabbitMQ (ADR-019), and Notification consumes them asynchronously.

Consequences:

- A booking is confirmed only after Wallet accepts the charge, so no booking is confirmed unpaid.
- Booking depends on Wallet being available to pay or cancel.
- Notifications are eventually consistent, and an event can be lost if publishing fails after the database write.

ADR-021: Consul for Service Discovery and Health-Aware Routing

Status: Accepted.

Context: Scaling a service means running several copies of it. Fixed service addresses cannot tell the gateway which copies exist or which are healthy, so a crashed copy would keep receiving requests. Options considered: Consul as a service registry, relying on fixed service names alone, and a hand-maintained list of addresses in the gateway.

Decision: Use Consul as the service registry. Every service registers itself and is health-checked, and the API gateway chooses among the healthy instances of a service itself instead of using a separate load balancer. If Consul is unavailable, the gateway falls back to the fixed service names.

Consequences:

- Replicas can be added or can fail without changing gateway configuration, and a failed instance stops receiving traffic automatically.
- No separate load balancer is needed, but the gateway carries the routing logic and is still a single instance, so it is the next bottleneck when the system scales.
- Consul is one more component to run, and the single-node local setup is not highly available.
- The gateway stays available when Consul is down, but without health-aware routing.
- Calls between services still use fixed names, so they do not benefit from discovery yet.

ADR-022: Prove Scalability with a Load Test

Status: Accepted.

Context: The non-functional requirement is that tutor search scales when more Tutor Service instances are added. A scalability claim needs measured evidence. Options considered: arguing from the design alone, a manual check with a few requests, and an automated load test.

Decision: Verify the requirement with an automated load test of tutor search through the gateway, using k6 (open source, from Grafana Labs). The test compares one and three Tutor Service instances against fixed limits: 95th-percentile latency under 500 ms and errors under 1%. Each instance has a capped CPU so the result does not depend on how powerful the laptop is.

Consequences:

- Three instances gave about twice the throughput of one (2,170 against 1,105 requests per second), with a 95th-percentile latency of 73.5 ms and no errors, so the requirement is met.
- The gain is 2x rather than 3x because the single gateway reached about 100% CPU, which confirms the gateway as the next component to scale.
- Stopping one instance during the test caused no failed requests, which supports ADR-021.
- The numbers come from one laptop running every container, so they show the trend between one and three instances, not production capacity.

7. Service Design and Architecture

7.1. Actors

Actor	Type	What the actor initiates
Student	Human	Searches for tutors, books and pays for lessons, tops up the wallet, attends lessons, reviews completed bookings.
Tutor	Human	Applies for tutor capability, publishes a listing, subjects and availability, teaches booked lessons, withdraws earnings.
Admin	Human	Reviews tutor applications and listing changes, moderates flagged review content.
PromptPay payment provider	External system	Confirms or rejects an incoming top-up payment.
Bank payout provider	External system	Executes an outgoing transfer to a saved payout account and reports its outcome.
Online meeting provider	External system	Provisions the meeting room for a confirmed online lesson.
Email provider	External system	Delivers notification messages.

7.2. Business Use Cases

ID	Use case	Actors	Business outcome
UC-01	Apply to Become a Tutor	Student supporting: Admin	A student submits an application with documents; the admin decision either grants tutor capability to the same account or records a rejection reason.
UC-02	Publish Subjects and Availability	Tutor supporting: Admin	Listing and verification-document changes pass admin review, and the tutor's subjects, rates and dated 30-minute slots become publicly discoverable and bookable.
UC-03	Discover and Pay for a Booking	Student	A student finds a tutor's subject, selects a continuous block of slots and pays from the wallet; the booking is confirmed immediately with subject, schedule and price fixed.
UC-04	Top Up the Wallet	User supporting: Payment provider	A confirmed PromptPay payment increases available balance exactly

ID	Use case	Actors	Business outcome
			once and appears in transaction history.
UC-05	Deliver and Complete a Lesson	Student and Tutor supporting: Meeting provider	Participants join the confirmed online lesson; attendance completes the booking, records pending tutor earnings and enables the review action.
UC-06	Cancel a Booking	Student or Tutor	The booking is cancelled after the participant sees the applicable policy, lesson access is removed, and any eligible refund is recorded in the student's wallet.
UC-07	Request a Payout	User supporting: Payout provider	Available funds are reserved and transferred to a saved payout account; the outcome and resulting balance are recorded in transaction history.
UC-08	Review a Completed Booking	Student supporting: Tutor, Admin	A review is published against the completed booking's tutor and subject and contributes to visible rating summaries; tutors may reply and admins may moderate flagged content.
UC-09	Resolve a Payment Dispute	Admin supporting: User	A reported payment issue receives a recorded decision, and any authorised financial adjustment appears exactly once in wallet history.

7.3. From Use Cases to Operations and Services

Use case	Operations it requires	Service that owns those operations
UC-01 Apply to Become a Tutor	`submitTutorApplication()`, `cancelTutorApplication()`, `reviewTutorApplication()`	Tutor Service
UC-02 Publish Subjects and Availability	`submitListingRevision()`, `reviewListingRevision()`, `manageSubjectOffering()`	Tutor Service
	`openAvailability()`	Booking Service
UC-03 Discover and Pay for a Booking	`searchTutors()`, `getTutorProfile()`, `getSubjectOffering()`	Tutor Service
	`getAvailability()`, `createBooking()`, `payBooking()`	Booking Service
	`chargeLessonPayment()`	Wallet Service
UC-04 Top Up the Wallet	`topUpWallet()`, `getWallet()`	Wallet Service
UC-05 Deliver and Complete a Lesson	`getLessonAccess()`, `recordAttendance()`, `completeBooking()`	Booking Service
	`creditTutorEarning()`	Wallet Service
UC-06 Cancel a Booking	`cancelBooking()`	Booking Service
	`refundBooking()`	Wallet Service
UC-07 Request a Payout	`requestPayout()`, `listTransactions()`	Wallet Service
UC-08 Review a Completed Booking	`submitReview()`, `replyToReview()`, `flagReview()`, `moderateReview()`, `getRatingSummary()`	Review Service
	`getBooking()`	Booking Service
UC-09 Resolve a Payment Dispute	`openDispute()`, `reviewDispute()`	Wallet Service
	`getBooking()`	Booking Service
All use cases above	`sendNotification()`, `updateNotificationPreferences()`	Notification Service

Notification is listed once at the end rather than repeated: every use case that changes a visible outcome ends by calling it.

7.3.1 Why these five services

- **Tutor Service** groups qualification, the subject catalogue and discovery because all three describe one thing: what a tutor is allowed to offer publicly. Search reads catalogue data directly, so keeping it inside removes a duplicated read model and a synchronisation problem that buys nothing at this size.
- **Booking Service** groups availability, the booking lifecycle and lesson delivery because a lesson is a confirmed booking being delivered. Separating them would put the completion decision on one side of a network call and the booking state it updates on the other, with no independent business reason for the split.
- **Wallet Service** owns every money movement, including dispute remedies. Financial correctness depends on a single writer and a single audit trail; a separate dispute service would have to reach into wallet data to be useful.
- **Review Service** stays separate because reviews are student-authored content with their own eligibility and moderation rules. A tutor must not be able to change them through the service that owns their own listing.

- **Notification Service** stays separate because it is the only component that talks to an external delivery provider, and a slow or failed delivery must never affect a business outcome that has already been decided.
- **Sizing check.** With the names removed, each box still answers a distinct question - who may teach, what is booked and delivered, where the money is, what others say about it, and who was told. No box holds most of the business logic, and none exists only to make the architecture look like microservices.

7.4. Service-Operations-Collaborators

An operation is written as its caller sees it. A collaborator is another service whose operation must be invoked to complete the operation on the left; where no collaborator is listed, the service completes the work from its own data.

Service	Operations	Collaborators
Tutor Service Owns tutor eligibility, the public offering, and search over it	• `submitTutorApplication()` • `cancelTutorApplication()` • `reviewTutorApplication()` • `submitListingRevision()` • `reviewListingRevision()` • `manageSubjectOffering()` • `getSubjectOffering()` • `searchTutors()` • `getTutorProfile()`	• Booking Service `getAvailability()` • Review Service `getRatingSummary()` • Notification Service `sendNotification()`
Booking Service Owns tutor availability, the booking lifecycle, and lesson delivery	• `openAvailability()` • `getAvailability()` • `createBooking()` • `payBooking()` • `cancelBooking()` • `getLessonAccess()` • `recordAttendance()` • `completeBooking()` • `getBooking()`	• Tutor Service `getSubjectOffering()` • Wallet Service `chargeLessonPayment()` • Wallet Service `creditTutorEarning()` • Wallet Service `refundBooking()` • Meeting Adapter • Notification Service `sendNotification()`
Wallet Service Owns every money movement and its audit trail, including disputes	• `getWallet()` • `topUpWallet()` • `chargeLessonPayment()` • `creditTutorEarning()` • `refundBooking()` • `requestPayout()` • `openDispute()` • `reviewDispute()` • `listTransactions()`	• Booking Service `getBooking()` • PromptPay Adapter • Payout Adapter • Notification Service `sendNotification()`
Review Service Owns reviews, replies and moderation of flagged content	• `submitReview()` • `replyToReview()` • `flagReview()` • `moderateReview()` • `getRatingSummary()`	• Booking Service `getBooking()`
Notification Service Owns delivery preferences and delivery attempts	• `sendNotification()` • `updateNotificationPreferences()`	• Email Adapter

7.4.1 Adapters to external systems

Adapters keep provider-specific APIs out of the services' business logic. They hold no business state of their own.

Adapter	Owned by	External system	Purpose
Meeting Adapter	Booking Service	Online meeting provider	Provision the meeting room and issue participant access.
PromptPay Adapter	Wallet Service	PromptPay payment provider	Request a top-up payment and receive its confirmed outcome.
Payout Adapter	Wallet Service	Bank payout provider	Submit an approved payout and receive its outcome.
Email Adapter	Notification Service	Email provider	Deliver a notification message and report the attempt.

7.5. Architecture Diagram

An arrow means “A calls B”: the tail is the service that initiates the request. Response paths are not drawn. Each service owns its own data store, listed inside the service box; no service reads another service’s database directly - it calls that service’s API instead.

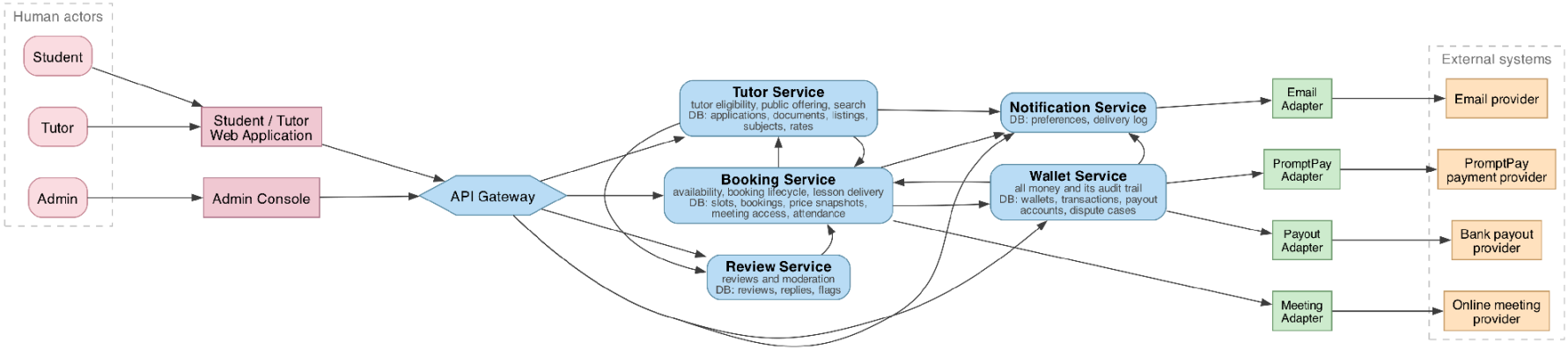


Figure 1 - Tutor Matcher service architecture, version 1. Pink: human actors and their clients. Blue: internal services and the API gateway. Green: adapters. Orange: external systems.

Version 2 of the architecture adds Consul as a service registry. Every service registers itself in Consul, and the API Gateway asks Consul for the healthy instances of a service before it forwards a request (see ADR-021). Services still call each other by fixed names. Tutor Service can run as several identical instances, and the gateway shares requests between them.